Enhancing Robot Navigation with a Low-Cost LIDAR and Absolute Localization Algorithm for small micro-controllers

Coupe de Robotique 2024 - Team "Mars Attacks!"

June 23, 2024

Abstract

This paper presents a computationally efficient algorithm for real-time pose estimation using 2D LIDAR data, optimized for small micro-controllers in the EUROBOT competition. Implemented on the Mars Attacks! robot, the algorithm significantly improves navigation and task performance by leveraging the playing area's unique features for accurate localization. Performance results demonstrate enhanced accuracy and robustness, making it suitable for cost-effective robot localization in constrained environments.

1 Introduction

One of the key challenges in mobile robotics is accurately determining the robot's pose in a given environment, which is crucial for effective navigation and task execution. Various algorithms have been developed for pose estimation using LIDAR data. Classical approaches, such as the Iterative Closest Point (ICP) algorithm, align point clouds by minimizing the distance between corresponding points. More recent methods leverage machine learning techniques to enhance accuracy and robustness, but often at the cost of significant computational resources, making them unsuitable for small micro-controllers.

This paper presents a novel, computationally efficient algorithm designed to process point cloud data from a 2D LIDAR sensor in real-time, optimized specifically for small micro-controllers. This algorithm is tailored for use on the EUROBOT playing area, ensuring effective pose estimation with limited computational power and memory.

We structured the paper as follows: First we describe the specific context of the EUROBOT competition for which we developed this algorithm and second we list the challenges of using a low-cost LIDAR and how to address these challenges. We then detail the design of the algorithm followed by the implementation and corner cases we encountered when applying the algorithm in practice. Finally we provide our conclusion and an outlook of future work in this area.

1.1 Holonomic robot special case

The Mars Attacks! robot, designed by our team, is a fast, holonomic robot equipped with four Mecanum wheels, enabling omnidirectional movement. While this design offers exceptional maneuverability, it poses significant challenges for accurate navigation due to the inherent complexities in tracking the robot's position and orientation. Traditional dead-reckoning methods, which rely on wheel and gyro odometry, often fail to provide the necessary precision for tasks such as gripping or dropping elements at specific locations on the playing area.

Our proposed algorithm addresses these challenges by integrating real-time LIDAR data with the robot's odometry to enhance pose estimation. It significantly improves the robot's ability to navigate and perform tasks with high precision.

The effectiveness of our algorithm was demonstrated during the Breizh ty Geek II and Coupe de Robotique d'Ile de France, where the Mars Attacks! robot exhibited markedly improved performance in both navigation and task execution. This advancement highlights the potential of our approach to overcome the limitations of dead-reckoning in holonomic robots, ensuring reliable and accurate operation in dynamic environments.

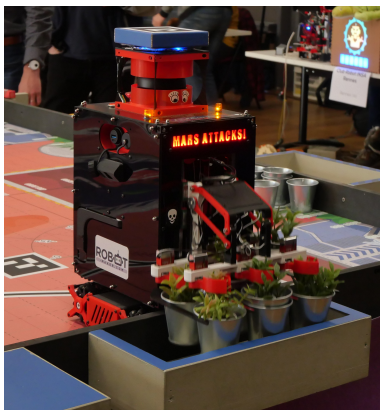


Figure 1: Mars Attacks! robot

2 EUROBOT Playing Area and Convention

The playing area is a rectangular plan of 3000 mm by 2000 mm with borders of 70 mm height and 22mm thick on each outer side. Orientations are TRIGO/CCW and World Reference Frame origin is at the center of the playing area.

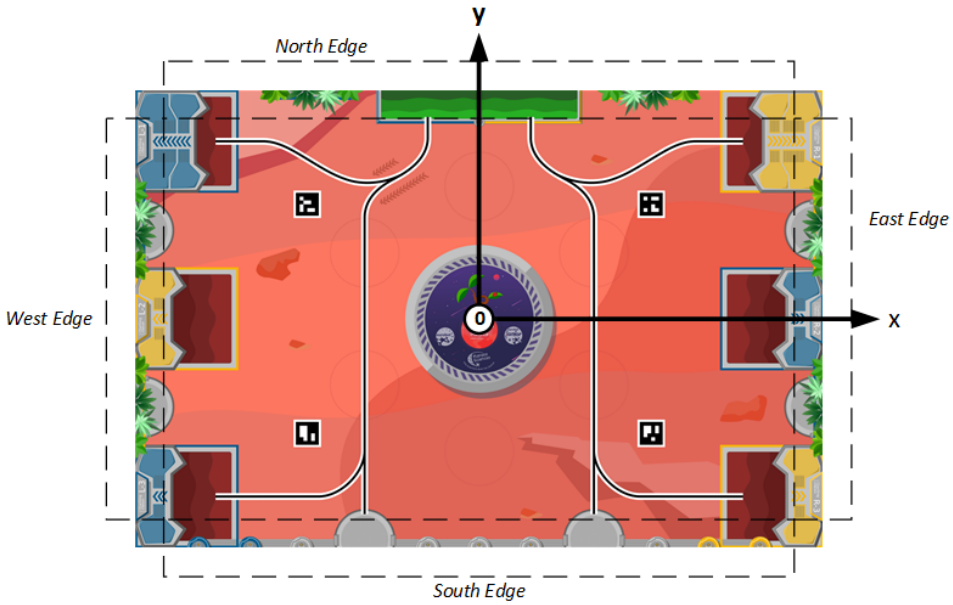


Figure 2: EUROBOT playing area

3 Shortcomings of low-cost LIDAR

We used the LD06 LIDAR which is available for less than 100 USD. This particular LIDAR provides ten scans of 360° per second at a resolution of about 450 points per scan (10HZ). While the relative low resolution has not posed challenges the relative low speed poses a challenge for a fast moving robot. Assuming the robot moves at a speed of 1 m/sec then the position of the robot will have changed by 10 cm between the first and the last point of a full 360° scan. We describe below how we compensate when calculating the point cloud of a full scan. Other LIDAR in the same price range expose similar challenges.

4 Design of the Algorithm

This section describes step by step, the proposed algorithm from LIDAR data acquisition to robot's pose correction.

4.1 Point Cloud Acquisition and Transformation

Transform the LIDAR scan distances issued by the sensor into a Cartesian point cloud in the World Reference Frame (WRF), based on the actual (though potentially inaccurate) robot pose. This transformation is the first crucial step in processing LIDAR data for real-time pose estimation. By converting the raw distance measurements into a point cloud in the world reference frame, the algorithm sets the stage for subsequent processing and pose estimation steps. By incorporating the robot's linear and angular velocities into the pose estimation during the LIDAR scan, the final point cloud compensates for the robot's movement, resulting in a more accurate representation of the environment.

4.1.1 LIDAR Data Capture

The LIDAR provides an array of distance measurements at different angles. Continuously capture raw distance data from the 2D LIDAR sensor.

4.1.2 Pose Estimation During LIDAR Data Reception

Estimate the robot's pose (x_t, y_t, θ_t) at the time each LIDAR measurement is received. Here, x_t and y_t are the robot's estimated position, and θ_t is its estimated orientation. This accounts for the robot's motion during the LIDAR scan. The pose estimation can be obtained by integrating the robot's velocity $(\dot{x}, \dot{y}, \dot{\theta})$ over time.

4.1.3 Polar to Cartesian Coordinate Transformation

Convert the polar coordinates (distance d and angle α) obtained from the LIDAR sensor into Cartesian coordinates in the robot's local reference frame (LRF).

$$x_{lrf} = d \cdot \cos(\alpha)$$
$$y_{lrf} = d \cdot \sin(\alpha)$$

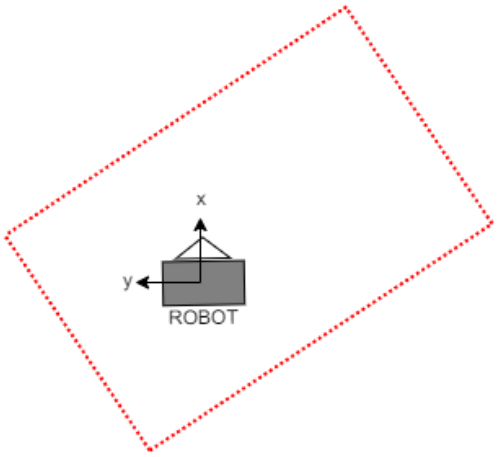


Figure 3: LIDAR points in the robot local reference frame

4.1.4 Pose Integration and Transformation to World Reference

Transform the LIDAR point cloud from robot local reference frame (x_{lrf}, y_{lrf}) to the world reference frame (WRF), using a translation and a rotation based on the robot's estimated pose (x_t, y_t, θ_t).

$$\begin{vmatrix} x_{wrf} \\ y_{wrf} \end{vmatrix} = \begin{vmatrix} x_t \\ y_t \end{vmatrix} + \begin{vmatrix} \cos(\theta_t) & -\sin(\theta_t) \\ \sin(\theta_t) & \cos(\theta_t) \end{vmatrix} \cdot \begin{vmatrix} x_{lrf} \\ y_{lrf} \end{vmatrix}$$

4.1.5 Point Cloud Construction

Aggregate all the transformed points to form the complete point cloud in the world reference frame.

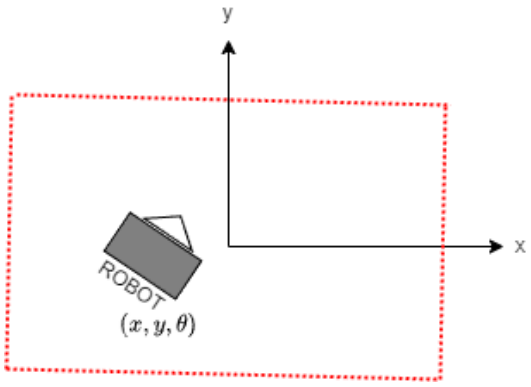


Figure 4: LIDAR point cloud in the world reference frame

4.2 Preprocessing the Point Cloud and Segmentation

Filter the point cloud to remove noise and irrelevant points, ensuring the data is clean and suitable for accurate pose estimation, specifically focusing on segmenting points that are near the edges of the EUROBOT playing area. Since the playing area is a rectangular table with borders, and the point cloud is in the world reference frame, a lot of points are expected to be near the edges. Segment these points by identifying those that are within a certain distance from the table borders.

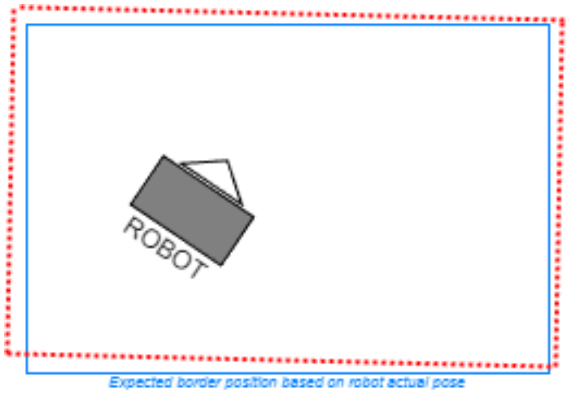


Figure 5: LIDAR points near the expected table borders

4.2.1 Edge Detection for Table Borders

Define an axis aligned boundary box (AABB) for each edge of the table: North, East, South, and West borders. The dimensions of the box are defined using the dimensions of the table (3m long and 2m wide) and an arbitrary distance margin (typically a few centimeters). Then, segment the points that fall within these boxes. Up to four group of points, corresponding of North, East, South and West edge are kept, and other points are just discarded.

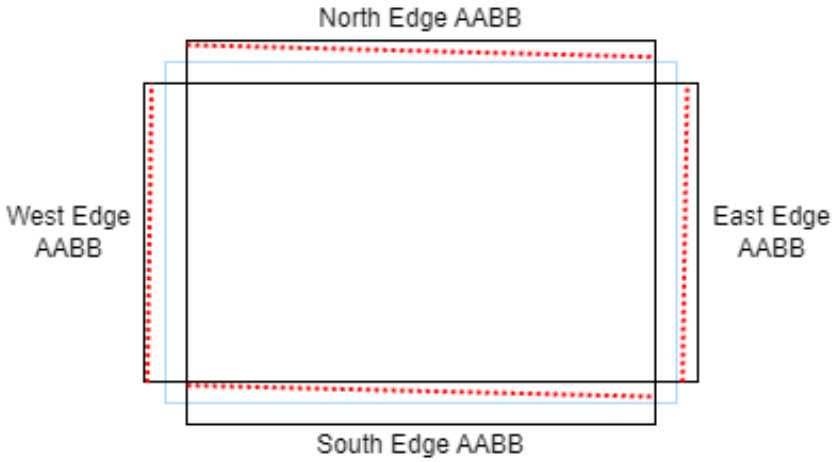


Figure 6: LIDAR point cloud segmentation using AABB on each edge of the table

4.2.2 Superposing Segmented Point Clouds from Parallel Edges

Utilize the symmetry of the horizontal (North,South) and vertical (East,West) edges of the table, to translate and superpose the segmented point clouds from parallel edges. This step helps to improve the consistency and accuracy of point cloud data for pose estimation.

Define the Translation Vector Calculate translation vectors that translate points from the edges to the center of the reference frame. Given the length and width of the table, these vectors are straightforward:

$$t^{NorthEdge} = \begin{vmatrix} 0.0 \\ -1.0 \end{vmatrix}, t^{EastEdge} = \begin{vmatrix} -1.5 \\ 0.0 \end{vmatrix}, t^{SouthEdge} = \begin{vmatrix} 0.0 \\ 1.0 \end{vmatrix}, t^{WestEdge} = \begin{vmatrix} 1.5 \\ 0.0 \end{vmatrix}$$

Translate the Edge Points Apply the translation vectors to the points from the north, east, south, and west edges to center and align them accurately.

Combine the Point Clouds Superpose the translated North and South segmented point clouds, resulting in a combined "North+South" segmented point cloud. Superpose the translated East and West segmented point clouds, resulting in a combined "East+West" segmented point cloud.

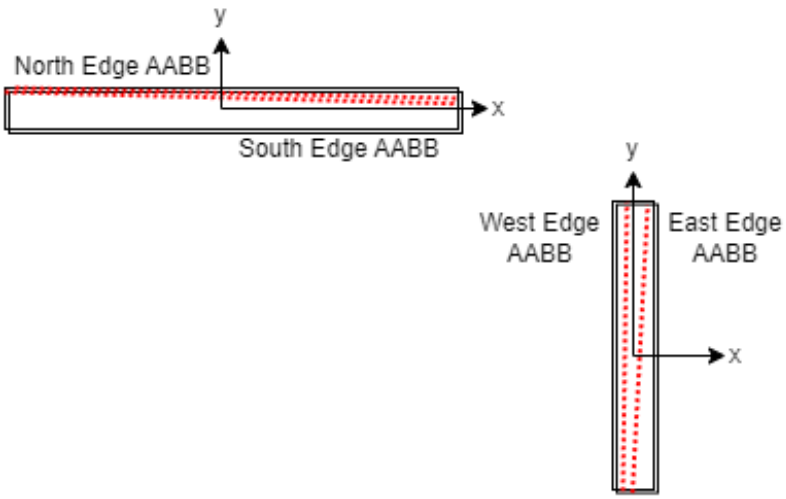


Figure 7: North-South combined point clouds —— East-West combined point clouds

4.3 Estimating Distance and Heading Angle Errors using Linear Regression

Use the combined point cloud from the North and South edges to perform linear regression and estimate the distance and heading angle errors of the robot relative to the horizontal edges of the table.

Use the combined point cloud from the East and West edges to perform linear regression and estimate the distance and heading angle errors of the robot relative to the vertical edges of the table.

Fit Lines using Linear Regression Perform linear regression on each combined point clouds to find the best-fit lines $y = a.x + b$. The slope of these lines represents the heading angle error, and the intercept gives information about the X or Y distance offsets. To avoid singularity when performing linear regression on the combined "East+West" point cloud (vertical edge), apply a 90° rotation onto those points before regression. Rotation matrix is straightforwards :

$$Rotation^{EastWest} = \begin{vmatrix} 0 & 1 \\ -1 & 0 \end{vmatrix}$$

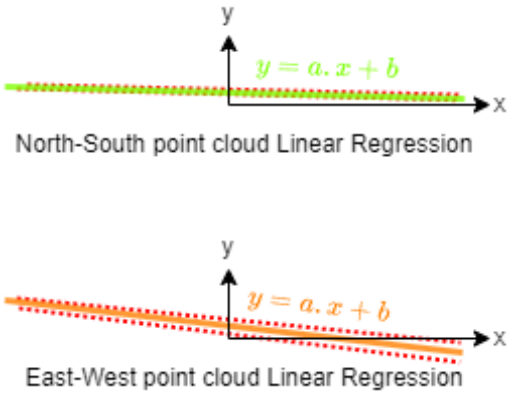


Figure 8: Linear Regression of both North-South and East-West (90° rot.) combined point clouds

Linear regression should be computed from a point cloud with a minimum number of points. An arbitrary threshold is defined, depending on the performance of the LIDAR sensor (angular resolution, measurement confidence). When the number of points if less than this threshold, (a, b) are set to zero.

Calculate X and Y Distance Errors The distance errors (x_{error}, y_{error}) can be determined by evaluating the distance of the fitted line, taking into account the intercept.

Calculate Heading Angle Errors The heading angle error θ_{error} can be derived from the slope of the best-fit line. Convert the slope to an angle in radians.

Edge	Number of points	Best-Fit Line	x_{error}	y_{error}	θ_{error}
North-South	N^{NS}	$y = a^{NS}.x + b^{NS}$	0	b^{NS}	$\arctan(a^{NS})$
East-West	N^{EW}	$y = a^{EW}.x + b^{EW}$	$-b^{EW}$	0	$\arctan(a^{EW})$

Merging Heading Angle Errors Calculate the overall heading angle error by merging the heading angle errors obtained from the North-South and East-West point clouds, using a weighted mean, based on the number of points in each combined point cloud.

$$\theta_{error} = \frac{N^{NS}.\arctan(a^{NS}) + N^{EW}.\arctan(a^{EW})}{N^{NS} + N^{EW}}$$

4.4 Pose Correction and Final Pose Estimation

Using the errors obtained from the LIDAR scan, update the robot's pose estimation to provide an accurate position and orientation.

The computed $(x_{error}, y_{error}, \theta_{error})$ represent the corrections needed to align the robot's estimated pose (x_t, y_t, θ_t) with the actual pose derived from the LIDAR data.

$$\begin{vmatrix} x_t \\ y_t \\ \theta_t \end{vmatrix}^{accurate} = \begin{vmatrix} x_t \\ y_t \\ \theta_t \end{vmatrix} + \begin{vmatrix} x_{error} \\ y_{error} \\ \theta_{error} \end{vmatrix}$$

Repeat the algorithm at each new LIDAR scan to refine the pose estimation over time. This helps in dynamically adjusting the robot's position and orientation as it moves.

5 Implementation

5.1 Hardware

The Mars Attacks! robot is equipped with a LD06 LIDAR sensor, strategically positioned approximately 6 cm above the playing surface, with a clear 360° field of view. We mounted the LIDAR upside down because the laser beam will then point slightly lower at the edges, thus reducing the risk that the LIDAR measures beyond the edges of the table. There are still situation where this can happen and we show below how we can deal with these situations.

The robot's pose estimation is updated at a high frequency of 1 kHz, leveraging the speed feedback from RoboMaster C610 Brushless Motor ESCs and a forward kinematic algorithm tailored for the Mecanum drive train. The robot's orientation is tracked using an Inertial Measurement Unit (IMU), which provides yaw gyroscope data. To maintain accuracy, a continuous drift-calibration algorithm is applied.

The integration of these components allows the robot to perform real-time pose corrections based on LIDAR data. Each time a new LIDAR frame of 12 measurements is received (approximately 380 frames per second), the system transforms the polar coordinates into Cartesian coordinates using the current estimated pose of the robot. Upon completion of a full scan (10 scans per second), the proposed algorithm computes the necessary pose corrections. This setup ensures that the robot can dynamically adjust its position and orientation, maintaining high precision in navigation and task execution despite the constraints of small micro-controllers.

5.2 Source Code

```cpp
// One-Pass Absolute Localization algorithm
size_t localisation(
        pose_t & pose_correction_output,                // Output: (x,y,w) computed pose correction
        const std::vector<point2f_t> & input_point_cloud,// Input data: (x,y) LIDAR point cloud in world reference frame
        float margin = 0.025f,                          // Input parameter: max distance between a point and an edge
        size_t threshold = 50                           // Input parameter: minimum number of points
    ) // return the number of points used for computing the returned pose correction
{
    // Constants for AABB computations
    float const x_west  { margin - 1.5f };
    float const y_south { margin - 1.0f };
    float const x_east  { -x_west };
    float const y_north { -y_south };
    // Initialize sums for linear regression
    float sum_x[2]  {0.0f};
    float sum_y[2]  {0.0f};
    float sum_x2[2] {0.0f};
    float sum_xy[2] {0.0f};
    size_t point_count[2] {0};
    // Segment point cloud and perform linear regression on-the-fly
    for (const auto & pt : input_point_cloud)
    {
        float xt { 0.0f }, yt { 0.0f };         // For translation and rotation
        if (y_south < pt.y && pt.y < y_north)         // Filter point using AABBs
        {
            if (pt.x < x_west) // West AABB
            {
                xt = pt.y;
                yt = -1.5f - pt.x;
                sum_x[1] += xt;
                sum_y[1] += yt;
                sum_x2[1] += xt * xt;
                sum_xy[1] += xt * yt;
                ++point_count[1];
            }
            else if (pt.x > x_east) // East AABB
            {
                xt = pt.y;
                yt = 1.5f - pt.x;
                sum_x[1] += xt;
                sum_y[1] += yt;
                sum_x2[1] += xt * xt;
                sum_xy[1] += xt * yt;
                ++point_count[1];
            }
        }
        else if (x_west < pt.x && pt.x < x_east) // South AABB
        {
            if (pt.y < y_south)
            {
                yt = pt.y + 1.0f;
                sum_x[0] += pt.x;
                sum_y[0] += yt;
                sum_x2[0] += pt.x * pt.x;
                sum_xy[0] += pt.x * yt;
                ++point_count[0];
            }
            else if (pt.y > y_north) // North AABB
            {
                yt = pt.y - 1.0f;
                sum_x[0] += pt.x;
                sum_y[0] += yt;
                sum_x2[0] += pt.x * pt.x;
                sum_xy[0] += pt.x * yt;
                ++point_count[0];
            }
        }
    }
    // Finalize linear regressions
    float a[2] {0.0f}, b[2] {0.0f};
    for (size_t index = 0; index < 2; ++index)
    {
        if (point_count[index] > threshold)
        {
            float const denominator { (static_cast<float>(point_count[index]) * sum_x2[index]
                                       - sum_x[index] * sum_x[index]) };
            float const k { 1.0f / denominator };
            a[index] = (static_cast<float>(point_count[index]) * sum_xy[index] - sum_x[index] * sum_y[index]) * k;
            b[index] = (sum_y[index] * sum_x2[index] - sum_x[index] * sum_xy[index]) * k;
        }
    }
    // Compute x, y, w errors from linear regression equations
    pose_correction_output.x = -b[1];
    pose_correction_output.y =  b[0];
    pose_correction_output.w = ( static_cast<float>(point_count[0]) * atan2f(a[0], 1.0f)
                               + static_cast<float>(point_count[1]) * atan2f(a[1], 1.0f) )
                               / (point_count[0] + point_count[1]);
    // Return the number of points (confidence)
    return (point_count[0] > threshold ? point_count[0] : 0) + (point_count[1] > threshold ? point_count[1] : 0);
}
```

5.3 Performances

The proposed algorithm demonstrates remarkable computational efficiency and accuracy on resource-constrained hardware. When implemented on a STM32F4 microcontroller running at 160 MHz, the algorithm processes approximately 450 LIDAR points and computes the robot's pose correction in just 100 µs.

For more complex scenarios, we implemented an extended version of the algorithm that performs four successive passes, each refining the pose estimation. This advanced version runs on a Teensy 4.1 board, which features a faster processor. Despite the increased complexity, this implementation completes the overall pose correction in approximately 200 µs, maintaining real-time performance.

The efficiency of our algorithm allows it to be seamlessly integrated into the fast-paced operations, providing accurate and reliable pose corrections without overburdening the microcontroller. This performance is critical for maintaining precise navigation and task execution.

5.4 Match logs

A fragment of the localization log was extracted from the Mars Attacks! robot after a qualification match, providing insight into the algorithm's performance in a dynamic and competitive environment. During this match, the robot was navigating at a high speed of 0.8 m/s with a rotational speed of 167°/s. The LIDAR point cloud captured during this period includes not only the borders of the table but also various objects such as pots, plants, and the adversary robot positioned at the opposite base.

The logs demonstrate the algorithm's capability to accurately differentiate and process relevant environmental features despite the presence of numerous obstacles and high-speed movement. The LIDAR data reveals the effectiveness of the segmentation and filtering processes in isolating the table borders for reliable pose estimation. The continuous pose corrections ensure that the robot maintains precise navigation and positioning, even in the presence of dynamic elements and potential collisions.

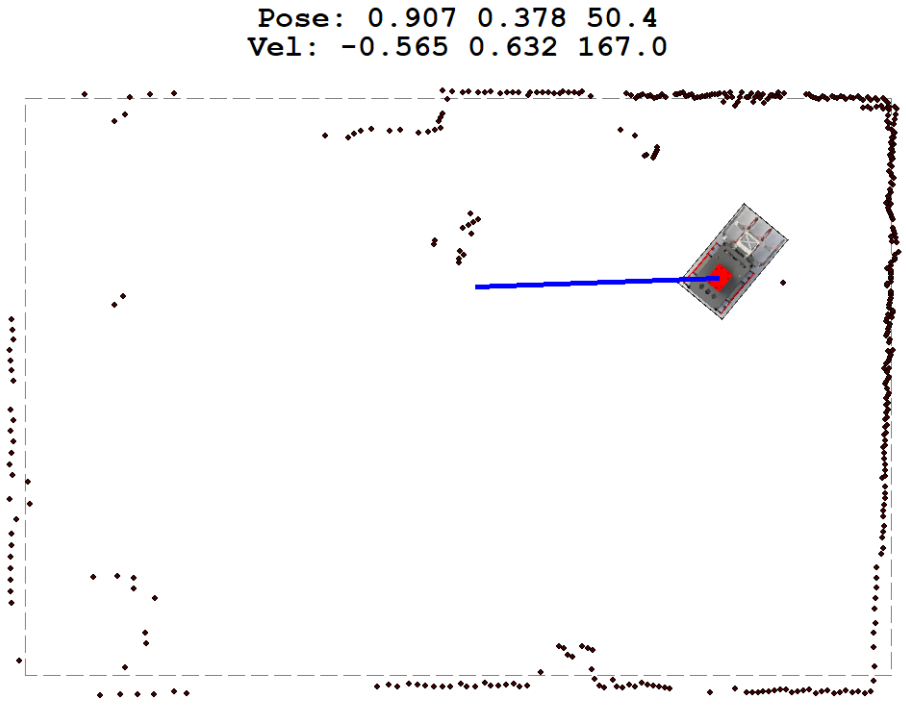


Figure 9: LIDAR point cloud in the world reference frame while fast robot move

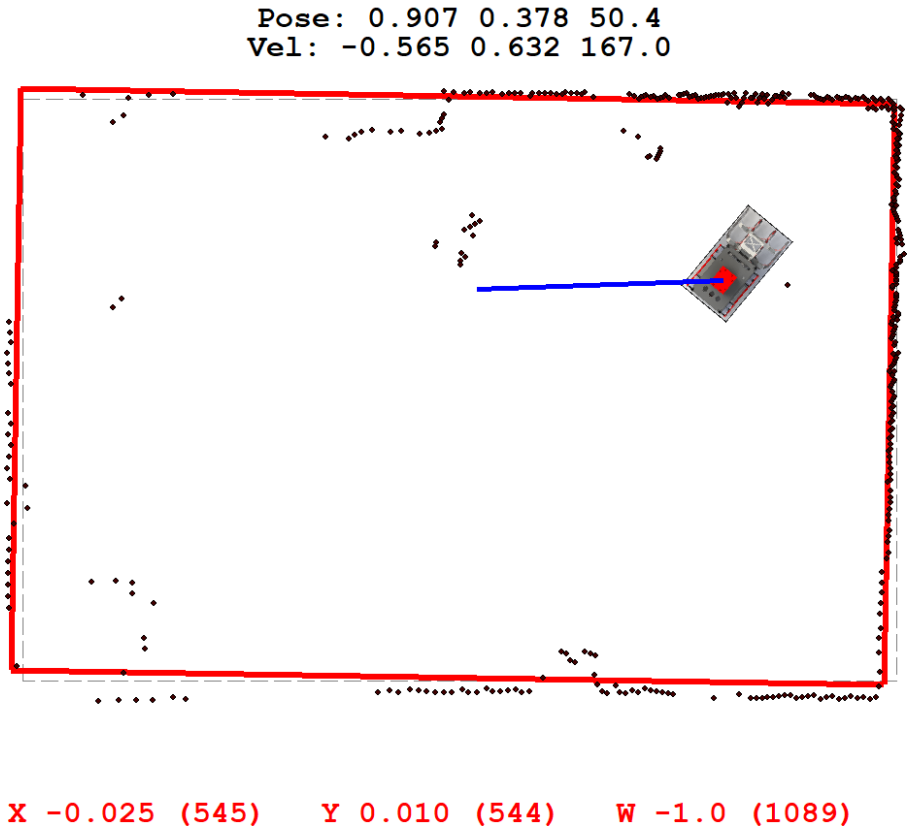


Figure 10: Computed x, y, θ (m,m,deg) pose correction while fast robot move (with number of points)

6 Future Work

A promising direction for future improvement of the algorithm involves performing multiple passes on the same LIDAR point cloud with varying margin values (e.g., 0.100, 0.025, 0.010 meters). By testing different margins, the algorithm can adapt to various levels of noise and point cloud density, potentially enhancing pose estimation accuracy. The pose correction with the highest confidence score from these multiple passes can be selected for final pose estimation. Alternatively, the different results can be merged using a weighted mean based on their respective confidence scores, allowing for a more robust and reliable pose correction.

Additionally, an improvement could be achieved by performing a two-stage pass approach:

- First Pass with High Margin: Conduct an initial pass using a high margin value to compute a preliminary pose correction. This helps in dealing with significant noise and larger errors.
- Second Pass with Low Margin: Translate and rotate the point cloud using the initial pose correction obtained from the first pass. Then, perform a second pass with a lower margin value to refine the pose correction. This approach allows for finer adjustments and improves accuracy by leveraging the cleaner point cloud.

Additionally, future work could explore:

- Dynamic Margin Adjustment: Implementing a dynamic adjustment of the margin values based on real-time analysis of the LIDAR data quality and environmental conditions.
- Cancellation of Abnormal Pose Corrections: This involves implementing a dynamic verification process to prevent the application of incorrect pose corrections, particularly in scenarios where the robot experiences tilting during high acceleration.
- ROS2 Integration: Develop a ROS2 node that incorporates an optimized driver for the LD06 LIDAR sensor, featuring speed compensation, alongside the proposed absolute localization algorithm.

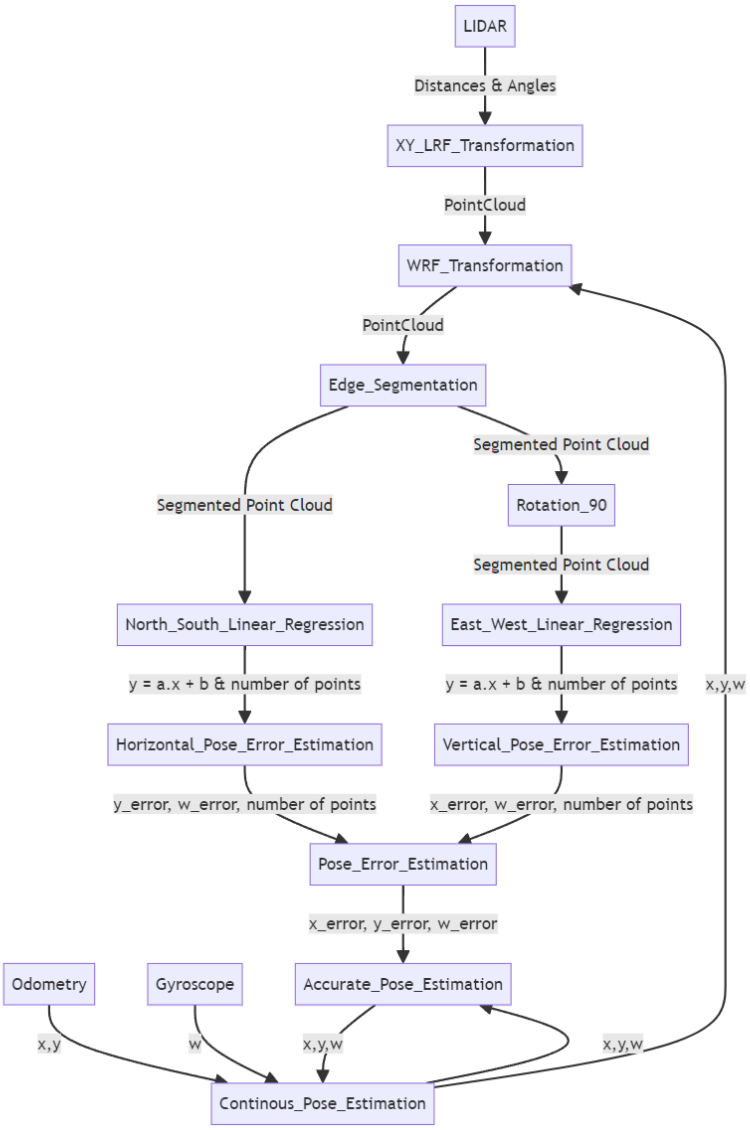


Figure 11: Flow diagram of the proposed algorithm

7 Conclusion

This paper introduces a computationally efficient algorithm for real-time pose estimation using 2D LIDAR data, optimized for small micro-controllers, specifically for the EUROBOT competition. Implemented on the Mars Attacks! robot, the algorithm integrates LIDAR data with odometry to significantly enhance navigation and task performance. Despite the constraints of low-cost LIDAR and limited computational resources, the algorithm achieves high accuracy and robustness, demonstrating its potential for cost-effective robot localization in constrained environments. Future work will focus on further improving accuracy through multi-pass processing and dynamic margin adjustments, as well as integrating the algorithm into broader applications beyond the competition context.

8 Team



From left to right, Horst DUMCKE, Patrick DUPUTZ, Nicolas MIRAULT, and Jonathan QUILLES.

9 Evolutions

9.1 Vigibot

In 2018 [1], an "Orthogonal" SLAM algorithm is developped for Vigibot, using a RPLIDAR A2 and a PIC32 @ 80MHz.

9.2 Coupe de Robotique 2022

In 2022 [2], the LD06 LIDAR replaces the RPLIDAR A2 and the algorithm is implemented in a robot of the TDS-Team 72.

References

[1] Vigibot, "Demo SLAM and Vigibot remote control by Vigirobotics," 2018. [Online]. Available: Démonstration du SLAM et du système de téléopération par internet Vigibot réalisé par Vigirobotics.. [Accessed: June 22, 2024].

[2] TDS-Team 72, "Implementation of SLAM algorithm using LD06 LIDAR," Coupe de Robotique 2022. [Online]. Available: Coupe de France de Robotique 2022 - Série 5. [Accessed: June 22, 2024].